

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:** SHOPEZ: E-commerce Application
- **Team Members:** KOTA MEHER MADHURI (Lead Full-Stack Developer)
- **Affiliation:** SmartBridge Virtual Internship Program

2. Project Overview

ShopEZ is a modern, responsive full-stack e-commerce web application designed to offer an effortless online shopping experience. It simplifies the user journey from viewing catalog items to final billing, and provides robust, fault-tolerant offline capabilities when MongoDB database servers are unreachable.

Key Features & Functionalities:

- Secure Authentication: JWT-based registration & login with local storage fallback.
- Interactive Catalog: Dynamic grid listing fallback catalog items instantly.
- Interactive Checkout Wizard: 4-step checkout form flow inside the Cart page.
- Flexible Payments: Supports Credit/Debit cards, UPI handles, and Cash on Delivery (COD).
- Inline Benefit Overlays: Interactive modal screens for Shipping, Security, and Returns.
- Infinite Scrolling Marquee: Dynamic horizontal scrolling ticker banner on the homepage.

3. Architecture

- **Frontend:** React.js (v19) utilizing component-driven architecture. State management is synchronized via global React Context Providers (AuthContext and CartContext) to persist checkout details.
- **Backend:** Node.js and Express.js REST APIs with route-level security, mongoose database connection controls, and in-memory mock fallback layers.
- **Database:** MongoDB cloud database. Handled with 'bufferCommands: false' to prevent buffering timeouts during database access errors.
- **Schemas:** User Schema (credentials & admin role), Product Schema (items details), and Cart Schema (holds current user checkout items list).

4. Setup Instructions

- **Prerequisites:** Node.js (v18+), MongoDB Atlas Database, and pm package manager.
- **Clone Project:** git clone <https://github.com/mehermadhuri/ShopEz-MERN.git>
- **Install dependencies:** Run 'npm install' inside the root, server/ and client/ directories.
- **Environment config:** Configure a server/.env file containing PORT=5000, MONGO_URI, and JWT_SECRET.

5. Folder Structure

- **Client Directory:** /src/components (Navbar), /src/context (Auth & Cart state), /src/pages (Cart, Products, Home, Login, Register, Shipping), /src/services (api.js)
- **Server Directory:** /config (db connection), /controllers (auth, cart, products logic), /models (schemas), /routes (endpoints)

6. Running the Application

- **Frontend start:** Run 'npm start' inside the client/ folder (runs on <http://localhost:3000>)
- **Backend start:** Run 'npm start' inside the server/ folder (runs on <http://localhost:5000>)

7. API Documentation

- **POST /api/auth/register:** Register account. Payload: {name, email, password}
- **POST /api/auth/login:** User sign-in. Payload: {email, password}
- **GET /api/products:** Fetches catalog list.
- **GET /api/cart/:userId:** Fetches active user cart.
- **POST /api/cart/add:** Adds product item to cart. Payload: {userId, product}
- **POST /api/cart/remove:** Removes item from cart. Payload: {userId, productId}
- **POST /api/cart/clear:** Clears cart on checkout success. Payload: {userId}

8. Authentication

User sessions are managed via SON Web Tokens (JWT). Passwords are securely hashed on registration using bcryptjs before storage. The client stores authentication state inside localStorage. If the database server is offline, authentication switches dynamically to browser local storage validation so the frontend remains fully functional.

9. User Interface

- **Scrolling Marquee:** An infinite horizontal scrolling loop showing Why Choose ShopEz values on the home page.
- **Checkout Wizard:** A multi-step checkout form flow for shipping details and payment entry.
- **Inline Modals:** Shipping, Returns, and Safety terms open as overlay dialogs directly inside the Cart page.

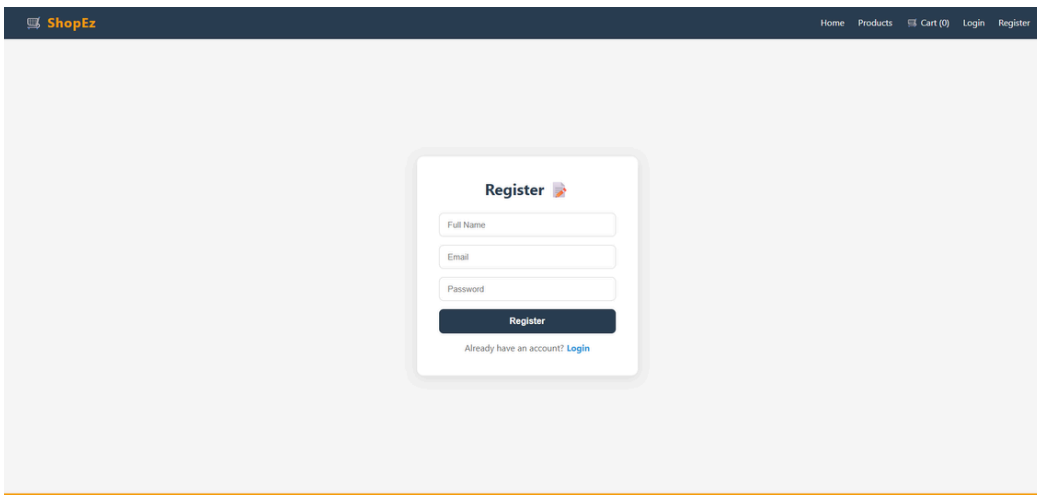
10. Testing

- **Frontend Tests:** Configured Jest assertions inside App.test.js to verify the ShopEz brand. Passed non-interactively.
- **API Verification:** Executed backend tests verifying response statuses 200/201 on auth endpoints.

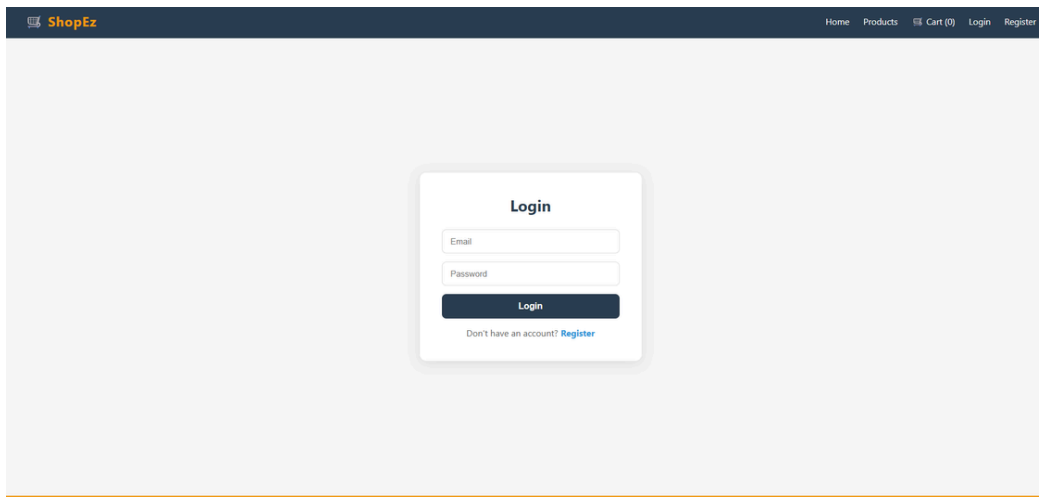
11. Screenshots or Demo

- **Screenshots:**

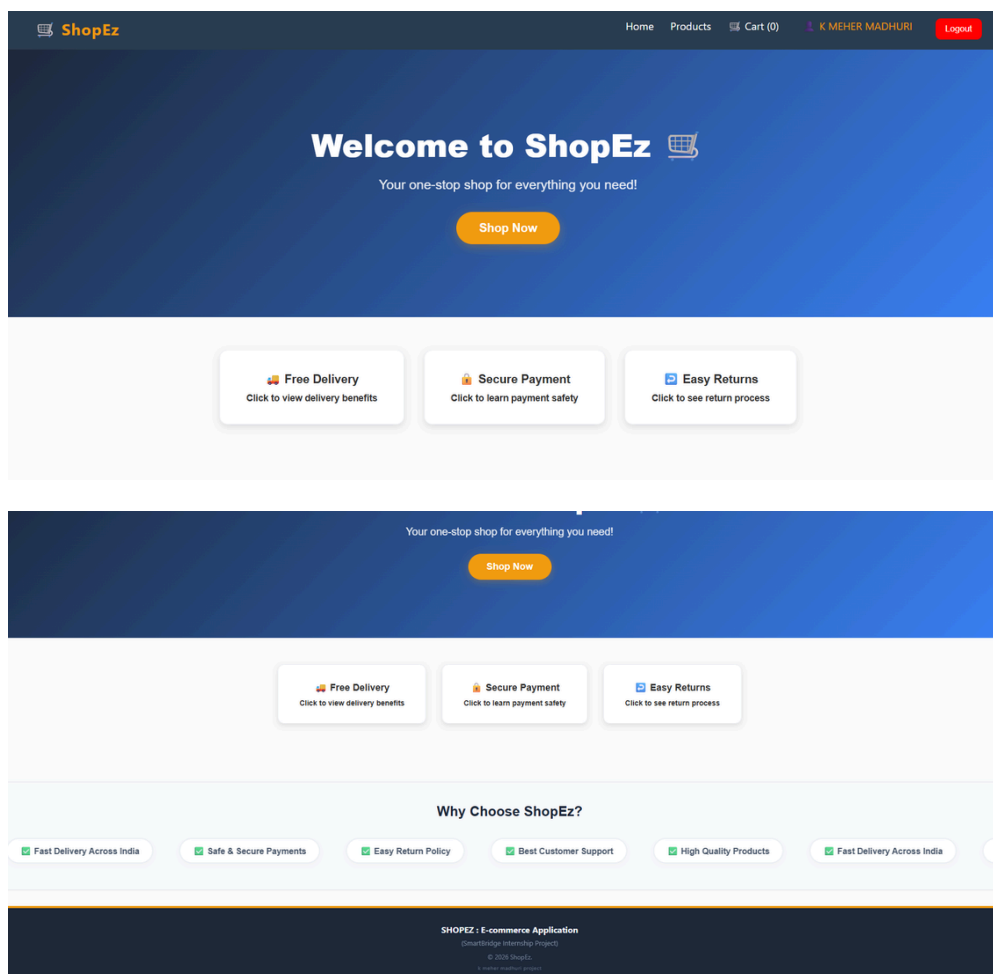
REGISTER PAGE



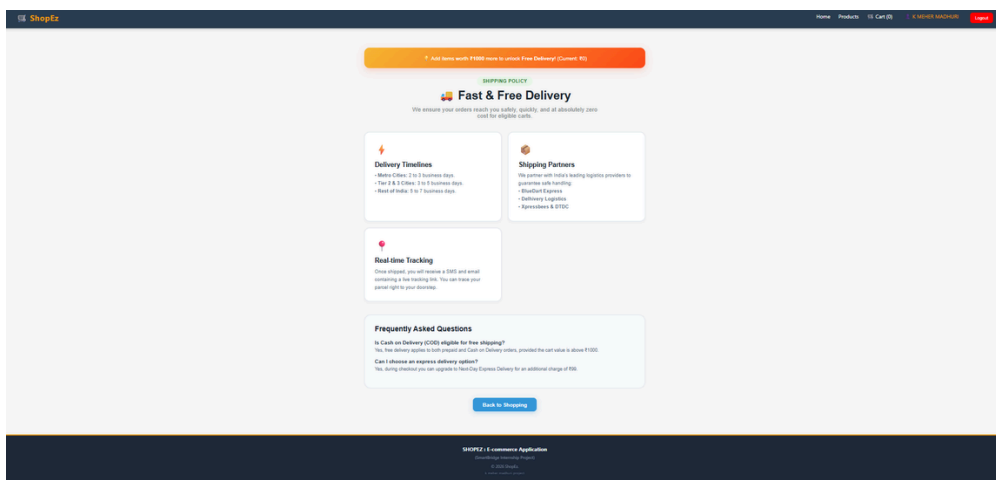
LOGIN PAGE



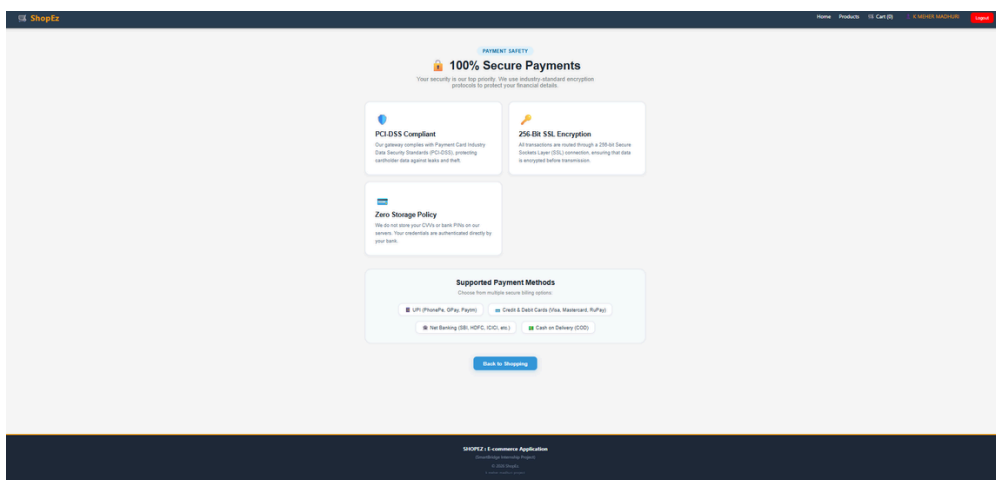
HOME PAGE



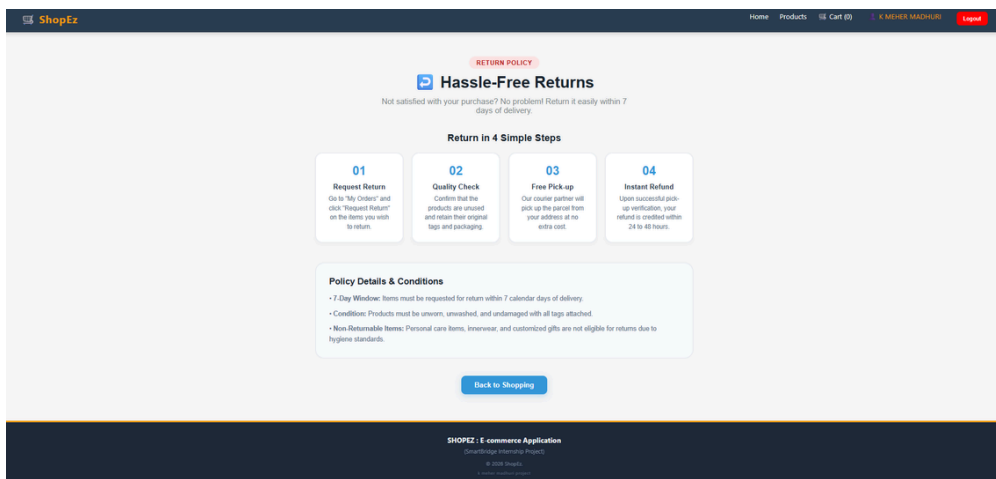
DELIVERY PAGE



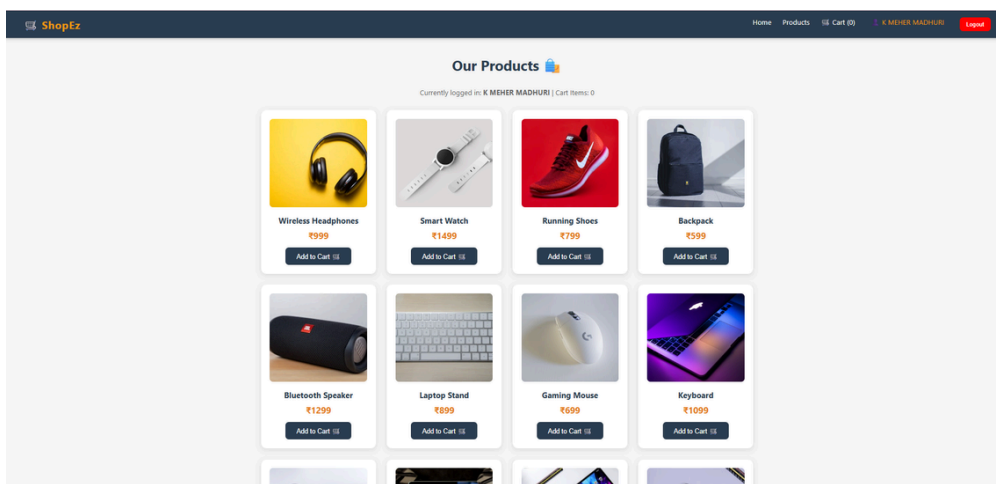
SECUREPAYMENTS PAGE



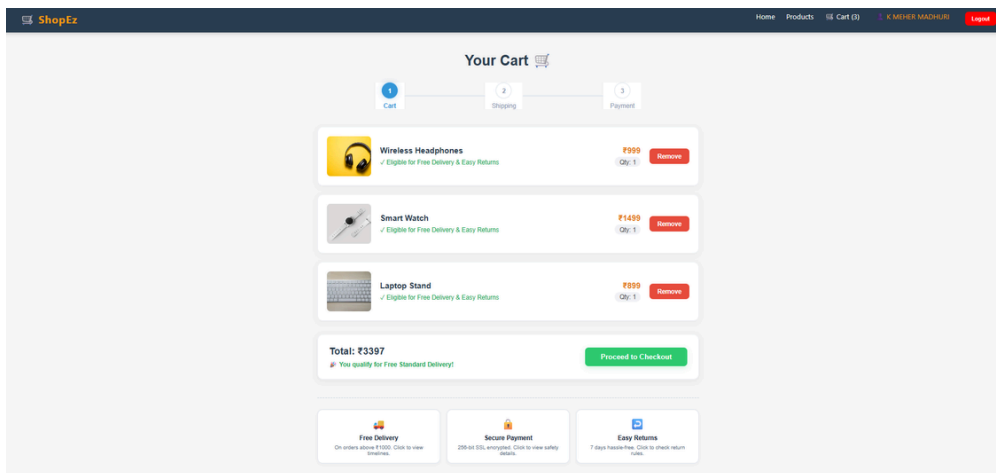
FREE RETURN PAGE



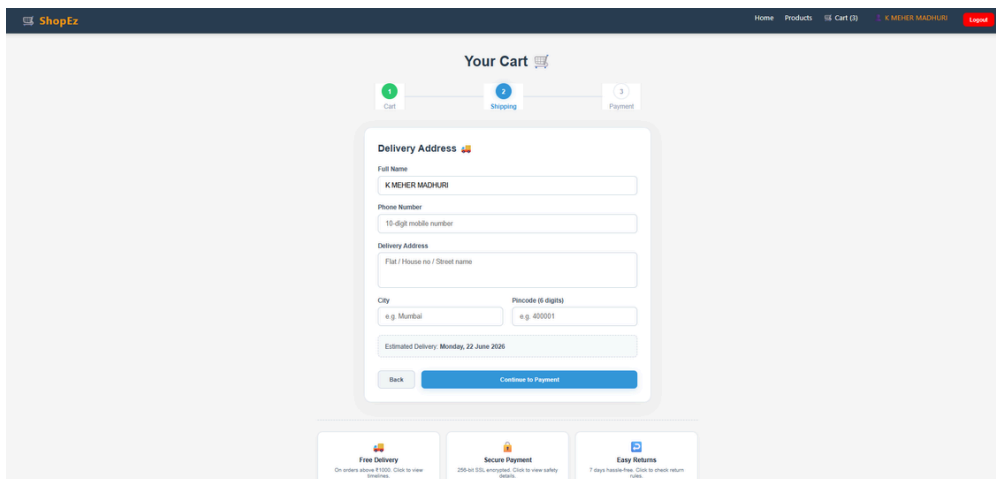
PRODUCTS PAGE



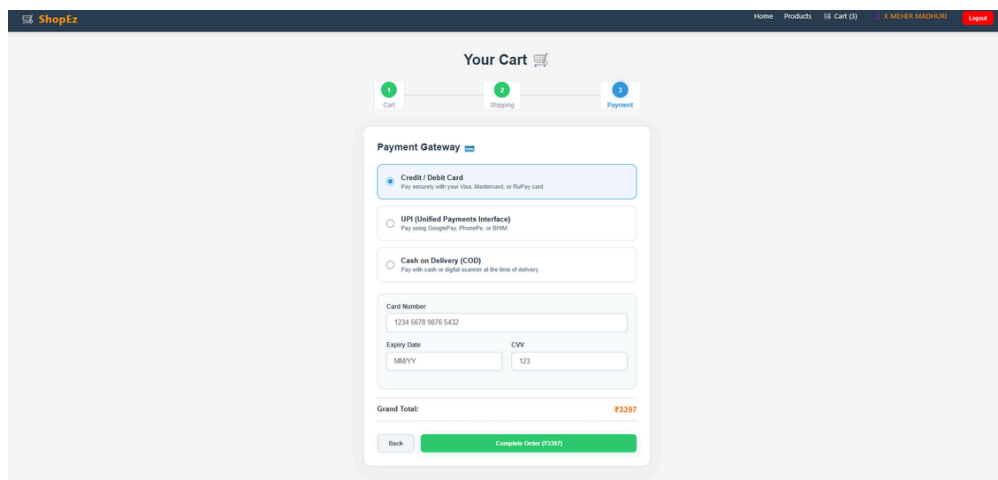
CART PAGE1



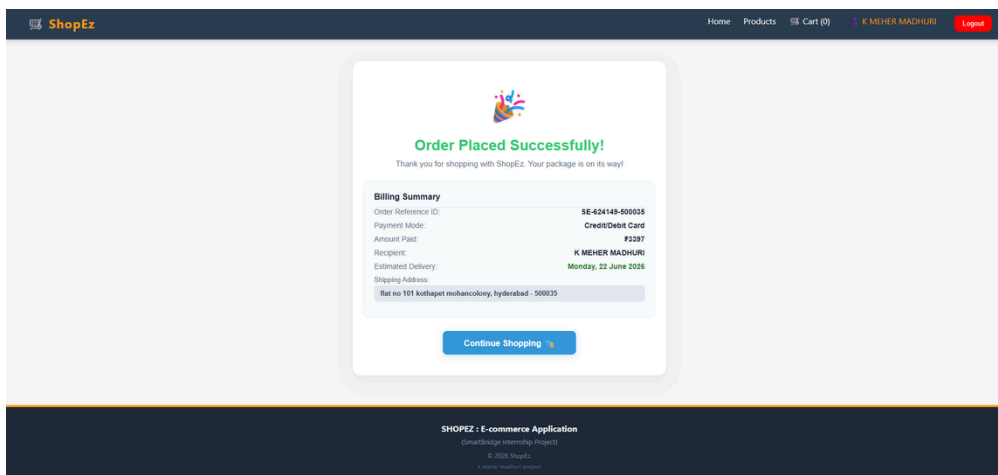
CART PAGE2



CART PAGE3



BILL PAGE



12. Known Issues

- **MongoDB connection:** Mongoose query buffering has been disabled. Connection drops are intercepted instantly, triggering fallback offline mode.

13. Future Enhancements

- **Live Payments:** Integration with Stripe or Razorpay API gateways.
- **Admin Inventory:** Enable store managers to add or delete items from the product grid directly via the UI.